הרצאה 3 – שפות תכנות 2027

מרצה: הדר פרנקל

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

היום נמשיך לעסוק בפרולוג ונעבור גם לדבר על ML.

Problem space search:

The pattern: generate and test

```
solution(S) :- candidate(S), ok(S).
```

candidate *generates* possible solutions one by one (on backtracking); **ok** *tests* them. A failed test sends Prolog back to the generator for the next candidate.

החוזקה הגדולה של פרולוג איננה אריתמטיקה, ישנו מסד נתונים עם עובדות ופעולות עליהן שאיתם אנחנו עובדים. בבעיות שבו יש מרחב חיפוש, אנחנו מתארים בצורה לוגית כיצד נראה פתרון ונותנים לפרולוג לחפש אותו. לרוב גם נשתמש בו בבעיות שממילא אנחנו מחפשים בהם בכל המרחב. הדפוס שנראה הוא זה משמאל: נגדיר פתרון solution(S), שיחפש את כל המועמדים שיכולים להיות פתרון ונבדוק שהם אכן יכולים להיות פתרון.

The Knapsack problem

Item	kg	Calories
bread	4	9200
pasta	2	4500
peanut butter	1	6700
baby food	3	6900

אנחנו רוצים לארוז לטיול, ובתרמיל שלנו יש 4 קילו (מס' שרירותי). המטרה שלנו היא לארוז כך שנעמוד בקיבולת וגם נארוז מאכלים שיתנו לנו כמה שיותר אנרגיה (למקסם את מס' הקלוריות). באופן כללי – יש תיק ומשקלים והמטרה היא למקסם את המשקלים תחת העובדה שעומדים בקיבולת. גישות חמדניות לא עוזרות לפתור את הבעיה. לדוגמה:

הבעיה היא בNPC – כלומר איננה ניתנת לפתרון פולינומי כרגע.

The plan: generate and test

Generate	a candidate knapsack: any *subsequence* of the pantry	→ **subseq**
Test	its total weight is at most the capacity	→ **weight**
	its total calories are at least some number *N*	→ **calories**

נגדיר פריט food(Name,weight,calories) להיות מזון עם שם, משקל ומס' קלוריות. וכן נחזיק במזווה עם הרבה פריטי מזון. תרמיל הוא תת רשימה של המזווה, והוא מועמד לפתרון טוב.

נתחיל מבעיה פשוטה יותר – עוד לפני שנפתור את בעיית האופטימיזציה, ננסה לפתור בעיית הכרעה. האם בכלל יש תרמיל עם כמות אנרגיה מעל N נתון? לרוב יותר קל להסתכל על בעיית ההכרעה, במקום בעיית האופטימיזציה.

```
% subseq(X,Y): X is Y with zero
% or more elements omitted
subseq([],[]).
subseq([Item|RestX], [Item|RestY]) :-
  subseq(RestX,RestY).      % keep Item
subseq(X, [_|RestY]) :-
  subseq(X,RestY).          % skip it
```

נתבונן בקוד הבא: נגדיר פרדיקט subseq להיות פרדיקט שמגדיר את כל תתי הרשימות של רשימה נתונה. בתחילה רשימה ריקה, לאחר מכן אם יש לנו רשימה ורשימה אחרת, אזי הרשימה מצד שמאל תהיה תת רשימה של הרשימה מצד ימין אם לאחר הורדת הפריט המשותף שלהן,

עדיין יש הכלה. האופציה השנייה היא להתעלם מהרכיב. כלומר אם נסכם: או זוג רשימות ריקות, או פריט משותף ואז מתקדמים בלעדיו ברקורסיה או שלא לוקחים אותו (המשותף) ומתעלמים ברקורסיה.

סה"כ בידינו פרדיקט שמחזיר את כל תתי הרשימות של רשימה נתונה. כיצד נעזר בו?

```
?- subseq(X,[1,2,3]).
X = [1,2,3] ;
X = [1,2] ;
X = [1,3] ;
X = [1] ;
X = [2,3] ;
X = [2] ;
X = [3] ;
X = [] ;
false.
```

ראשית כאשר נרצה לייצר את כל תתי הרשימות נשים משתנה X שיעשה זאת ויתן לנו את תתי הרשימות אחת אחרי השנייה:

בשלב הבא – עלינו לבצע בדיקה כי המשקל לא עובר את הקיבולת הנתונה, וכן שאנחנו מחזיקים יותר מN קלוריות:

בקוד משמאל: אנחנו מחשבים את סך המשקל של רשימה נתונה.
אם היא ריקה אז אפס, אחרת אנחנו מסתכלים רק על המשקל של פריט מזון ברשימה X ומתקדמים ברקורסיה על שאר הרשימה, כאשר מוסיפים את W לתוך המשתנה RestW.

```
% total weight of a list
weight([],0).
weight([food(_,W,_)|Rest], X) :-
  weight(Rest,RestW),
  X is W + RestW.
```

בקוד מימין: שוב, כאן אנחנו מחשבים את מס' הקלוריות הכולל.

```
% total calories of a list
calories([],0).
calories([food(_,_,C)|Rest], X) :-
  calories(Rest,RestC),
  X is C + RestC.
```

<u>להלן הפתרון לבעיית ההכרעה:</u>

הוא מקבל את התרמיל, קיבולת, מטרה N, ולKnapsack מוכנס בשלב הראשון תתי הקבוצות שנבחרו: הוא מחשב את המשקל ובודק שזה לא עולה על הקיבולת. ואז בודקים שמס' הקלוריות לפחות מעל ערך המטרה. אם כל זה מתקיים התשובה היא true.

```
% a knapsack within Capacity with at least Goal calories
knapsackDecision(Pantry,Capacity,Goal,Knapsack) :-
  subseq(Knapsack,Pantry),      % generate
  weight(Knapsack,Weight),      % test
  Weight =< Capacity,
  calories(Knapsack,Calories),
  Calories >= Goal.
```

כעת, נרצה לחזור לדון בבעיית האופטימיזציה. כעת נרצה לדון – כיצד נמצא את כל הפתרונות האפשריים, ונדע להכריע על בעיית האופטימיזציה? לשם כך נחזור לדיון באופן כללי:

findall(Template, Goal, List)

1. Find *all* solutions of **Goal** (by backtracking, as if we pressed ; again and again)
2. For each solution, record the value of **Template**
3. **List** is the list of all the records

<u>הפרדיקט findall:</u> פרדיקט שמוצא את כל הפתרונות כך שGoal הוא ערך המטרה שאנחנו מעוניינים לספק, List זה כל תתי הרשימות שאנחנו מחזירים ומיד נרחיב על Template.

ניתן לקרוא את זה כ – תכניס לתוך L את כל האיברים בTemplate שמקיימים את התנאי Goal.

לדוגמה: כאן במקום לקבל את הפתרונות אחד אחרי השני, מקבלים אותם ברשימה אחת פעם אחת.

One at a time

```
?- parent(kim,C).
C = holly ;
C = kent.
```

All at once

```
?- findall(C,parent(kim,C),L).
L = [holly, kent].
```

נרחיב על Template: נניח לצורך הדוגמה כי המטרה היא subseq(X,[1,2])

Template	Recorded for each solution	Resulting L
X	the value of X	[[1,2], [1], [2], []]
1	the constant 1, whatever X is	[1, 1, 1, 1] one 1 per solution, so it *counts* them
subseq(X, [1,2])	the whole goal, with X filled in	[subseq([1,2],[1,2]), subseq([1],[1,2]), subseq([2],[1,2]), subseq([],[1,2])]

- אם הוא X, אזי מה שישמר בתוך האאוטפוט הוא כל הX כך שX הוא תת רשימה.
- אם הוא 1, אזי: "תחזיר לי 1 עבור כל אחד שהוא פתרון עבור הרשימה הנתונה". כלומר נקבל 1 עבור כל תת קבוצה. פחות שימושי.
- אם הוא subseq(X,[1,2]): במקרה שכזה, הוא מחזיר רשימה של כל הפרדיקטים שהם מקיימים את הGOAL. כמו שניתן לראות בדוגמה.

נחזור לדבר על בעיית האופטימיזציה. לשם כך נגדיר עוד מספר פרדיקטי עזר:

בשלב הראשון, נחפש את כל **התרמילים החוקיים:** הם אלו שהמשקל שלהם הוא לא עולה על הקיבולת שלנו. התנאי הראשון לבדיקה הוא שהתרמיל תת רשימה של המזווה שלנו. לאחר מכן מחשבים את המשקל של התרמיל לתוך W, ובודקים שהוא לכל היותר הקיבולת.

```
legalKnapsack(Pantry,Capacity,Knapsack) :-
  subseq(Knapsack,Pantry),
  weight(Knapsack,W),
  W =< Capacity.

knapsackOptimization(Pantry,Capacity,Knapsack) :-
  findall(K, legalKnapsack(Pantry,Capacity,K), L), % all legal
  bestKnapsack(L,Knapsack).                        % the best
```

לאחר מכן, בעיית האופטימיזציה מוגדרת כך: מצא לי את התרמיל האופטימלי – כיצד? בתוך L תכניס את כל התרמילים החוקיים, אלו שמשקלם לא עולה על הקיבולת. ולאחר מכן – מצא את התרמיל הטוב ביותר.

כל שנותר הוא להגדיר את bestKnapsack: כיצד?

```
% start: the first one is the best so far
bestKnapsack([K|Others],Best) :-
  calories(K,KCal),
  bestFrom(Others,K,KCal,Best).

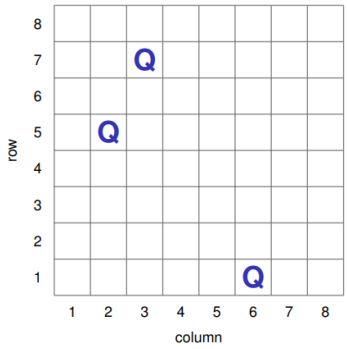
% bestFrom(Knapsacks,SoFar,SoFarCal,Best)
% 1. none left: SoFar is the answer
bestFrom([],SoFar,_,SoFar).
% 2. K at least as good: new best
bestFrom([K|Others],_,SoFarCal,Best) :-
  calories(K,KCal),
  KCal >= SoFarCal,
  bestFrom(Others,K,KCal,Best).
% 3. K is worse: keep the best so far
bestFrom([K|Others],SoFar,SoFarCal,Best) :-
  calories(K,KCal),
  KCal < SoFarCal,
  bestFrom(Others,SoFar,SoFarCal,Best).
```

המטרה שלנו היא להשוות בין כל התרמילים החוקיים, כל תרמיל חוקי הוא רשימה.

- בשלב הראשון, מסתכלים על האיבר הראשון. בתחילה הוא מוגדר להיות הכי טוב. אנחנו מחשבים את מס' הקלוריות שלו לתוך KCAL, הוא מס' הקלוריות המקסימלי שמצאנו עד כה. וBEST הוא התרמיל הכי טוב שמצאנו עד כה.
- ואז: בדיקה פשוטה למדי. (1): אם יש לנו רשימה ריקה, סיימנו. מי שבידינו הוא הכי טוב.

ההמשך – בדיקה פשוטה אם הוא גדול או קטן מהיעד וכו', ובסוף בBEST יש את התרמיל הכי טוב.

The Eight Queens Problem



בעיית שמונה המלכות מוגדרת באופן הבא.

- כל מלכה יכולה לזוז כיצד שהיא רוצה, ימינה שמאלה וכו'. שתי מלכות מאיימות זו על זו אם הן באותה שורה/עמודה וכו'.
- המטרה שלנו היא למקם 8 מלכות על לוח שחמט ריק כך שאף אחת לא מאיימת על אף אחת אחרת.
- נייצג מלכה ע"י הקורדינטה שלה – זוג סדור. C/R.

בתחילה נגדיר את הפרדיקט noAttack: מתי מלכה לא מאיימת על אף אחת ברשימה שלי? בהינתן רשימה ריקה של מלכות, אין לנו איומים. אחרת: אם יש לנו מלכה כלשהי C/R ואחת אחרת בראש רשימה: נרצה לבדוק ראשית שהן לא באותה שורה ולא באותה עמודה, וכן לא באותו אלכסון.

```
% noAttack(Queen,Others): Queen attacks none of the queens in Others
noAttack(_, []).
noAttack(C/R, [C1/R1 | Others]) :-
  C =\= C1,                          % not the same column
  R =\= R1,                          % not the same row
  abs(R1-R) =\= abs(C1-C),           % not the same diagonal
  noAttack(C/R, Others).
```

וכן נגדיר: לוח הוא חוקי, אם ורק אם אף מלכה לא מאיימת על אחת אחרת. לוח ריק הוא חוקי. מה הרעיון? לוקחים C כלשהו מתוך 1-8 וR כלשהו מ1-8 ובודקים שאין סכנה, ואם כן מציבים אותו.

```
% legal(Board): no queen on Board attacks another
legal([]).
legal([C/R | Others]) :-
  legal(Others),                  % the other queens are already legal
  member(C,[1,2,3,4,5,6,7,8]),    % choose a column for this queen
  member(R,[1,2,3,4,5,6,7,8]),    % choose a row for this queen
  noAttack(C/R, Others).          % test it against the others
```

מה יקרה אם נריץ את זה סתם? הוא ייתן לנו את כל הלוחות האפשריים:

```
?- legal(Board).
Board = [] ;
Board = [1/1] ;
Board = [1/2] ;
etc.
```

לכן מראש נגדיר את הלוח להיות בגודל 8 על 8.

```
?- Board = [_,_,_,_,_,_,_,_],
   legal(Board).
Board = [8/4, 7/2, 6/7, 5/3,
         4/6, 3/8, 2/5, 1/1] .
```

הקשר בין לוגיקה לפרולוג

כאשר אנחנו מבצעים שאילתא, אנחנו בודקים האם טענה זו נובעת מהעובדות והחוקים שיש בידינו. אם פרולוג עונה שכן: הוא מצא **הוכחה** לשאילתא שלנו.

ראשית, נזכר כי: $A \to B = \neg A \vee B$

כלומר: זה נכון או כשB מתקיים, או שA לא מתקיים. לכן: ע"י חוק

$$h : -b_1, b_2$$

שהוא למעשה $b_1 \wedge b_2 \to h$ יכול להיות מתורגם ע"י $\neg b_1 \vee \neg b_2 \vee h$. שכן:

$$\neg \left(b_1 \wedge b_2\right) \vee h = \neg b_1 \vee \neg b_2 \vee h$$

לכן, פרדיקט מתורגם לפרדיקט. אבל מה עם חוק? למשל:

$$\text{ancestor}\left(X, Y\right) : - \text{ parent } \left(X, Y\right). \Longrightarrow \neg \text{parent}\left(X, Y\right) \vee \text{ancestor}\left(X, Y\right)$$

נשים לב כי עלינו להוסיף גם $\forall X, Y$, שכן זה נכון לכל X וY.

נסכם:

1. עובדה מתורגמת לפרדיקט חיובי.
2. חוק פשוט מהצורה $h : -b_1, b_2$ מתורגם כמו שראינו.
3. אם יש לנו אוסף למשל $h : -b_1, b_2, b_3$ אזי הוא מתורגם ל

$$\neg b_1 \vee \neg b_2 \vee \neg b_3 \vee h$$

שוב זה נובע מהכלל כי לכל $n \in \mathbb{N}$:

$$h : -b_1, ..., b_n \equiv \neg \left(\wedge_{i=1}^{n} b_i\right) \vee h \equiv \vee_{i=1}^{n} \neg b_i \vee h$$

כיצד נחפש הוכחה לתוכניות שלנו? נדון בשיטה בשם רזולוציה.

נניח אוסף העובדות והחוקים הנ"ל

$$\text{rain. wet : } -\text{rain. sllipery: -wet.}$$

ואז נניח שאילתא: $-$ sllipery?

נרצה להוכיח שאכן מחליק. כיצד?

בפרולוג:

goal slippery, rule 3: new goal wet. rule 2: new goal rain. rule 1:no goals left: true.

בלוגיקה:

נניח בשלילה כי השאילתא היא שקר, כלומר לא מחליק. אזי, בהכרח גם לא wet, ולכן בהכרח לא rain - זו סתירה כי זו עובדה.

הגדרה 1. רובינסון: אם A, B, L ליטרלים המקיימים כי $A \vee L$ וגם $\neg L \vee B$ אזי נובע מכך כי $A \vee B$.

$$\frac{\text{A} \vee L \quad \neg L \vee B}{\text{A} \vee B}$$

כך הדוגמה הקודמת נעשית בפרולוג: אם הגענו למצב הריק ־ סתירה, אזי הצלחנו להוכיח את מה שניסינו להוכיח בתחילה. כלומר זה מאוד דומה ללוגיקה ־ הנחה בשלילה. שיטה זו נקראת **רזולוציה.**

$$\neg slippery \text{ and } \neg wet \vee slippery \text{ imply } \neg wet$$
$$\neg wet \text{ and } \neg rain \vee wet \text{ imply } \neg rain$$
$$\neg rain \text{ and } rain \text{ imply } \square \text{ (nothing left: false)}$$

מסקנה: הגעה לסתירה ־ השאילתא נכונה. אין סתירה ־ השאילתא לא נכונה.

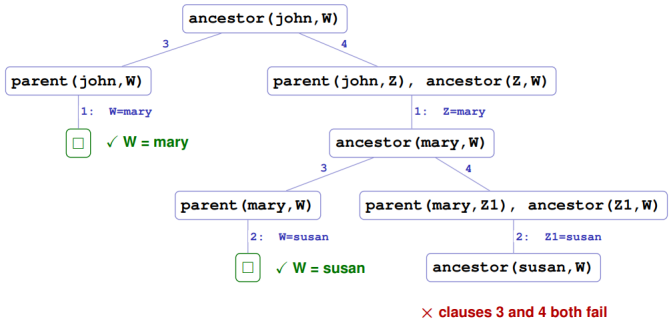
הגדרה 2. רזולוציה:

1. כותבים את התוכנית כclauses (חיבור של ליטרלים בסימן $\vee$) ומוסיפים $\neg G$ ("assume the query is false").
2. מבצעים רזולוציה שוב ושוב.
3. אם נגיע ל$\square$ אזי המסקנה היא סתירה: שאילתא נכונה. אם לא הגענו לסתירה: עושים באק־טראקינג ומנסים להגיע לענף אחר. אם לא נמצאה סתירה בכלל: השאילתא לא נכונה.

הגדרה 3. Horn Clauses הוא קלאוז שבו יש לכל היותר ליטרל חיובי אחד בלבד. כלומר, ללא סימן $\neg$.

מדוע פרולוג דורש Horn Clauses? כי מאוד נוח לו במקרה זה למצוא ליטרל ושלילת הליטרל, ולבטלם כיוון שאנחנו ב$\vee$.

דוגמה לבאק־טראקינג על שאילתא: כל ירוק משמעותה שהגענו לסתירה. אדום משמעותו שלא הצלחנו להגיע לסתירה.



נאותות: כל תוכנית פרולוג מתאימה לרצף לוגי כמו שתיארנו. עץ החיפוש מכיל את **כל** ההוכחות האפשריות. יתכן שההוכחה תהיה בענף אחרי ענף אינסופי, ולכן כך פרולוג לפעמים לא תמצא הוכחה ולכן **הסדר חשוב.**

מדוע פרולוג טובה, והשקענו בזה 3 הרצאות?

- טוב מאוד לבעיות בהן צריך לסרוק ולחפש את מרחב החיפוש.
- פרולוג היא הבסיס לכל מיני שפות שמתעסקות עם מסדי נתונים כמו SQL.

ובזאת סיימנו לדבר על Prolog.

0>0

ML

עד כה דיברנו על תכנות לוגי. נעבור לדון על תכנות פונקציונלי.

מדובר באחת השפות הכי פופולריות הפונקציונליות, נלמד SML: standard ML. נקרא לזה בכל זאת ML. מה היא מביאה שמיוחד?

- היא מסיקה מה הטיפוס של כל משתנה – לבד.
- יש פולימורפיזם: פונקציה יכולה לעבוד על סוגים שונים של משתנים כל עוד אנחנו עקביים.

הכל כאן פונקציות – יש לכל פונקציה ערך החזרה יחיד בהינתן קלט מסוים. **תמיד** נחזיר ערך.

- אין לנו אפשרות להשמות ועדכון משתנים.
- חזרתיות נעשית באמצעות רקורסיה, אין לולאות.

ML עושה אבולואציה ישירות. אם כתבנו 1+2*3 זה שומר את זה כ7. משמאל לימין: ML מקבל ערך, ומסיק את הטיפוס של המשתנה – כאן INT. הוא מכניס אותו לתוך משתנה בשם IT. המשתנה IT הוא משתנה כללי שמכניסים לתוכו דברים שלא הגדרנו להם משתנה.

```
- 1+2*3;
val it = 7 : int
- 1+2*3
= ;
val it = 7 : int
```

- כל שורה חייבת להסתיים בנקודה פסיק, אחרת ML לא יחזיר לנו תשובה.

לכל ערך בML ישנו טיפוס. לפני האבולוציה היא מבצעת בדיקת טיפוסים – ואז יודעת לומר לנו על שגיאה במידת הצורך. אלו הטיפוסים האפשריים:

Type	Values	Example constants
`int`	integers	`3`, `~5`
`real`	floating-point numbers	`3.14`, `~0.5`
`bool`	truth values	`true`, `false`
`char`	single characters	`#"a"`
`string`	text	`"hello"`

- על מנת לסמן איברים שליליים, למשל 5- אזי נכתוב זאת באמצעות טילדה: ~. 5~.
- אין דרך לחבר שלם עם ממשי מבלי לקבל שגיאה.
- תו מסומן ע"י # ומרכאות בין התו.
- מחרוזת מיוצגת עם מרכאות.

```
- 1234;
val it = 1234 : int
- 123.4;
val it = 123.4 : real
- ~3;
val it = ~3 : int
```

לדוגמה: מכניסים 1234 והוא כבר מזהה לבד את הטיפוס.

<u>אופרטורים:</u>

- יש +,-, *, /, div וכן mod. יש אסוציאטיביות בחישוב לפי ההיררכיה הבאה:

$$\{+,-\} < \{*,/,\mathbf{div},\mathbf{mod}\} < \{\sim\}$$

- שרשור רשימות מתבצע ע"י ^. כמו כן כל האופרטורים שאנחנו מכירים קיימים גם כאן. כמו בדוגמה משמאל:
- ניתן להשוות ע"י אופרטור רק שניים מאותו הטיפוס.

```
- "bibity" ^ "bobity" ^ "boo";
val it = "bibitybobityboo" : string
- 2 < 3;
val it = true : bool
- 1.0 <= 1.0;
val it = true : bool
- #"d" > #"c";
val it = true : bool
- "abce" >= "abd";
val it = false : bool
```

```
- 1 = 2;
val it = false : bool
- true <> false;
val it = true : bool
- 1.3 = 1.3;
Error: operator and operand don't agree
  [equality type required]
  operator domain: ''Z * ''Z
  operand:         real * real
```

- שוויון: שוויון חלש עובד על INT, לעומת זאת לא ניתן להשוות ממשיים. הסיבה היא כיוון שלא ניתן באמת לוודא כי שני reals שווים. הדרך לעשות את זה היא לבדוק $abs(x-y)<\varepsilon$
- לא שווה מסומן ע"י <>.
- אופרטורים בוליאניים:

ML	Meaning	In C
andalso	and	&&
orelse	or	||
not	not	!

בand,or הוא קודם בודק את צד שמאל ורק אם ישנה סיבה הוא בודק את צד ימין. למשל בAND אם השמאלי יצא 0 אין סיבה לקרוא את זה וזה לא יקרא.

- תנאים:

```
- if 1 < 2 then #"x" else #"y";
val it = #"x" : char
- if 1 > 2 then 34 else 56;
val it = 56 : int
- (if 1 < 2 then 34 else 56) + 1;
val it = 35 : int
```

תמיד מחזירים משהו. אם התנאי מתקיים תחזיר משהו, אחרת תחזיר משהו אחר.

ניתן גם להשתמש בערך החזרה כמו בדוגמה השלישית, ולהוסיף לו 1 למשל.

Function	Type	What it does	Example
real	int -> real	the same number, as a real	real 3 = 3.0
floor	real -> int	rounds down	floor 3.6 = 3
ceil	real -> int	rounds up	ceil 3.2 = 4
round	real -> int	rounds to the nearest	round 3.6 = 4
trunc	real -> int	drops the fraction	trunc ~3.6 = ~3
ord	char -> int	the character's code	ord #"a" = 97
chr	int -> char	the character with this code	chr 98 = #"b"
str	char -> string	a one-character string	str #"a" = "a"

פונקציות המרה: נשים לב שקוראים לפונקציה ללא סוגריים.

- כאשר מפעילים פונקציות, עושים זאת משמאל לימין. למשל f a b המשמעות היא: f(a) b. כלומר – הפעל את F על a ואת התוצאה הפעל על b.
- הפעלת הפונקציה זה הדבר הראשון שנעשה, קודם בסדר להכל (!). משמאל לימין.

```
- val x = 1+2*3;
val x = 7 : int
- x;
val it = 7 : int
```

משתנים: כאשר רוצים להגדיר משתנה עושים זאת באמצעות val. כותבים זאת מפורשות. כאן X הוא שם המשתנה. כאשר לא נעשה השמה למשתנה הוא יעשה זאת לבד למשתנה IT.

כאשר אנחנו נבצע למשל שורה כמו x=9 val, מדובר במשתנה חדש וזו לא השמה למשתנה x שהגדרנו קודם. בשורה זו אנחנו מגדירים משתנה **חדש**.